Beacon: Final Product Technical Report

A web-based training and support platform for university peer supporters

Joan Kariza · BSc Software Engineering, African Leadership University · Supervisor: Hubert Apana · November 2026

Repository: github.com/JoanK7/Beacon · Testing evidence: docs/testing.md

1. Scope Alignment and Feature Validation

The approved proposal (Chapters 1–3) defined ten functional requirements (FR1–FR10) and six non-functional requirements (NFR1–NFR6) for a mobile-first Progressive Web App that helps peer supporters at ALU train, refer and look after themselves. All ten functional requirements are implemented. Two items are partial and two are not implemented; each is justified below. Features added after supervisor and user feedback are marked **[added]**.

Planned feature	Intended purpose	Current status	Evidence (repository / tests)
FR1 Pseudonymous sign-up and session login	Secure access with minimal identifying data (no email, phone or real name)	Completed	*server/src/services/auth.service.ts*; httpOnly cookie, hashed tokens; integration tests "authentication (FR1)"
FR2 Training modules on the core competencies, with text, image and video	Standardise and refresh the in-person Solid Minds training	Partial	Six Solid Minds modules (PEERS model, mental health literacy, psychological first aid, boundaries, referral, self-care) with interactive blocks **[added]** and a Listen button. Image, video and audio are supported in the editor, but no partner media exists yet. **Safe self-disclosure** is not implemented (see justification).
FR3 Progress tracking	Show learners where they are; complete a module when every section is read	Completed	*training.service.ts* markSectionViewed (transaction, idempotent); journey path UI
FR4 Gamified quiz and badge	Engagement and knowledge retention	Completed	Question banks of 10 per module with 5 drawn per quiz, "new" vs "repeat" quizzes, and test-out **[added]**; points, levels, badges, streak and daily stories **[added]**; branching practice conversations **[added]**
Pre/post knowledge measure (RQ3)	Measure learning before and after training	Completed	Skill check with its own 12 questions, never revealed (*skill-check.service.ts*); normalised gain on the coordinator dashboard
5-item helping-behaviour confidence scale	Second RQ3 outcome	Completed	Co-designed in workshops with peer supporters; planned before data collection
FR5 Searchable directory, filterable by category and district	Fast, correct referrals	Completed	7 services verified with ALU and Solid Minds; category chips, on-campus flag, search across name, location and district; tap-to-call, email and web links
FR6 Support-session log with optional referral	Document referrals (RQ4)	Completed	Notes encrypted with AES-256-GCM; integration test checks ciphertext at rest
FR7 Self-care reflection	Prevent burnout among supporters	Completed	Mood check-in with encrypted text; anonymous community wall **[added]**
FR8 Anonymous feedback and SUS	Usability evidence (RQ4)	Completed	*feedback* table has no user column; SUS scoring unit-tested
FR9 Content administration	Partner-controlled content without code changes	Completed	Modules, quiz banks, directory, scenario visibility and community moderation; edits are never overwritten on deploy
FR10 Coordinator dashboard	Programme oversight without exposing individuals	Completed	Aggregates only, with groups under 3 people suppressed; learning-quality panel **[added]**
NFR1 Privacy and security (Law 058/2021)	Protect sensitive data	Completed	bcrypt, SHA-256 tokens, AES-256-GCM, RBAC, rate limits; key management and data-residency plan in *docs/security.md*
NFR2 Pages in under 3 s on mobile	Usable on campus networks	Completed	Largest contentful paint 2.5–2.8 s (simulated slow 4G, 4× slower CPU)

NFR3 Usability and accessibility	Inclusive, mobile-first	Completed	Lighthouse accessibility 100 on all pages; keyboard and reduced-motion support
NFR4 99% uptime	Reliability	Partial	Health endpoint and uptime-monitor plan ready; can only be measured once deployed (Render and Vercel configured)
NFR5 200 concurrent users	Scalability	Completed	200 connections: 0 errors, 99% of responses within 130 ms
NFR6 Maintainability, ≥60% API coverage	Long-term upkeep	Completed	97.0% line coverage; documented services; CI on every push
Kinyarwanda, email notifications, LMS integration	Reach and convenience	Not implemented	Out of scope in the proposal (§1.5); listed as future work

Justified deviations.

- **Safe self-disclosure.** It is not part of Solid Minds' curriculum or their list of core competencies. Writing clinical guidance without the partner would break the ethics commitment that all content is professionally reviewed. The prototype's placeholder module was archived.
- **JavaScript → TypeScript, and next-pwa → a hand-written service worker.** Typed API shapes catch broken data before users see it. next-pwa is unmaintained for Next.js 16.
- **Additions from feedback.** A peer counsellor found the modules "plain", which led to the interactive lesson blocks. The supervisor asked to stop memorising and rushing, which led to question banks, separate skill-check questions and server-side pace checks. The supervisor also asked for motivation and peer care, which led to the streak, stories and community wall. The free-text forum and peer pairing were deferred: they need a moderation and safeguarding plan.
- **Hosting.** Under Law 058/2021, Art. 50, storing real personal data abroad needs NCSA authorisation. The Render and Vercel deployment therefore holds demo data only until that authorisation is granted or the API moves to a Rwandan host (no code change needed).

Milestones. M1 Core platform, FR1–FR10 (28 Oct) → M2 Engagement and evidence: lesson blocks, compatibility and performance testing (29 Oct) → M3 Final version: supervisor feedback, verified content, security documentation (2 Nov). 20 commits follow Conventional Commits (feat, test, docs, ci), and each milestone is verified by CI.

2. Technical Implementation

Architecture. Three layers, as in the proposal. A **Next.js 16 / React 19 PWA** (TypeScript, Tailwind) renders the screens and proxies */api/** to an **Express 5 REST API**, so the session cookie stays first-party. The API is split into thin routes, business services and pure domain classes over **PostgreSQL 16** with versioned SQL migrations. Every request passes through helmet → CORS allow-list → a 100 kB JSON limit → *requireAuth* (cookie → hashed token lookup) → *requireRole* → zod validation → service → parameterised SQL → a central error handler that returns JSON and never a stack trace.

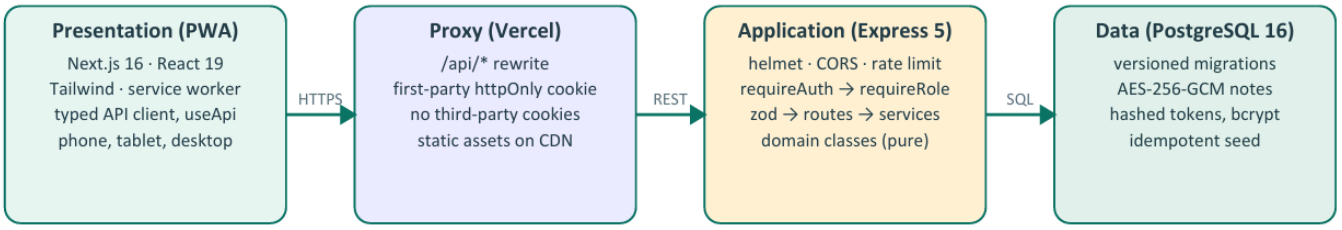


Figure 1. Beacon architecture and continuous integration.

Key backend–frontend flows.

- **Quiz attempt.** The browser asks to start an attempt. The server draws 5 questions (least-seen first), shuffles each question's options and stores the question ids and option orders, then returns questions without answers. On submit, the server maps the displayed positions back to the stored order and scores against what was actually shown. A row lock (FOR UPDATE) stops a double submit, and another user's attempt returns 404.
- **Reading pace.** Opening a module starts a server-side clock; marking a section read measures the elapsed time against the section's word count. Because the browser never reports timings, they cannot be faked.
- **Practice conversations.** The client sends only the path taken. The server replays it through the branching script and rejects impossible paths (400), so the insight score cannot be forged.
- **Data handling.** Notes are encrypted with AES-256-GCM (a random IV and tamper-detecting tag per value). The key is kept as a host secret with a backup in a password manager. Community posts never return names or ids and their times are rounded to the hour. Content updates are applied in place by version, keeping learners' progress and admin edits.



Figure 2. Interactive lesson: flip cards and a "Check yourself" question with kind feedback (FR2).



Figure 3. Practice conversation: an unhelpful choice shows Bea's feedback and can be retried; the server replays the path (FR4)

3. Testing, Validation and Performance

Strategy and tool	Why chosen	How conducted	Result
Unit (Jest)	Fast proof that pure logic is correct at its boundaries	109 tests: scoring, encryption, question selection, pace, streak, crisis detection, content and schemas	109/109 pass
Integration (Supertest + real PostgreSQL)	Proves each requirement end to end through HTTP and SQL	51 tests on a wiped, migrated and seeded database, the same as production	51/51 pass; 97.0% lines, 94.5% statements

Strategy and tool	Why chosen	How conducted	Result
End-to-end (Playwright)	Real user journeys in a real browser	10 journeys: the proposal's Figure 3.7 flow, lessons, practice, skill check, repeat/new quiz, story and streak, community, roles, reduced motion	All pass
Compatibility (GitHub Actions)	Peers use varied phones and browsers	7 profiles on 3 engines: Pixel 7, Galaxy S8 (360 px, low-end), iPhone 13, iPad, desktop Chrome, Firefox and Safari	70/70 runs pass
Performance (Lighthouse, autocannon)	NFR2 and NFR5 targets	Mobile with slow 4G and a 4× slower CPU; 200 concurrent connections for 20 s on 3 endpoints	LCP ≤2.8 s, accessibility 100; 0 errors, p99 ≤130 ms
Upgrade test (database migration)	Deploys must never lose progress	Previous release database plus learner data → new code, run twice	All 29 section ids, progress and referrals kept; idempotent
Usability (feedback and SUS form)	Content must engage peers	Peer counsellor and supervisor reviews; in-app SUS	Led to lesson blocks, quiz banks, streak, stories and community wall

Inputs and edge cases tested (expected behaviour shown):

- Auth: missing consent → 400; duplicate username in a different case → 409; a wrong password gets the same message as an unknown user, with timing equalised by a dummy hash.
- Quizzes: unanswered questions → 400; resubmission → 409; another user's attempt → 404; a repeat never awards a badge; a draw never exceeds the bank; least-seen questions win across 20 random seeds.
- Validation: a 31-character username, a *javascript:* or *http:* website, a 281-character note, or a reply outside the fixed set → rejected. Search treats *%* and _ literally; an impossible scenario path → 400.

- Safety and privacy: crisis phrases (including curly apostrophes) are flagged, while "exams are killing me" is not; 3 reports hide a post; posts expire after 7 days; coordinator averages from fewer than 3 people are hidden; deleting an account removes all linked data.

Improvements made after testing.

- **Compatibility** found that a 31-character test username was correctly rejected (the test data was fixed). It also found that typing before WebKit finished loading the page cleared the field. The tests now wait for the page to settle, and making forms keep early typing is the planned app fix.
- **Integration** showed that older skill-check results still contained the correct option, so the API now strips it from every response.
- **Performance**: after the stories card was added, the dashboard's Lighthouse score fell from 94 to 87, though its main content still appears in 2.8 s. Merging the dashboard's API calls is the next optimization. The landing page improved from 72–77 to 89.

Performance results (final-release run, 2 Nov 2026, GitHub Actions; full reports in the run's artifacts):

Page (Lighthouse, mobile)	Perf.	A11y	LCP
/ landing	89	100	2.5 s
/login	98	100	2.5 s
/dashboard	87	100	2.8 s
/learn/peers-model	96	100	2.7 s
/directory	97	100	2.7 s

API endpoint (200 connections, 20 s)	Req/s	p50	p99	Errors
/api/health (with DB query)	6,544	28 ms	59 ms	0
/api/modules (signed in)	1,840	107 ms	130 ms	0
/api/providers (search)	2,918	67 ms	81 ms	0

Analysis. Every page meets the NFR2 target of under 3 seconds on a throttled phone, and accessibility scores 100 everywhere (NFR3). Under 200 concurrent connections (NFR5), the heaviest endpoint, modules with per-user progress, answered 99% of requests within 130 ms with no errors. Shared CI machines vary between runs (an earlier run gave dashboard 94 and landing 72–77), so trends across runs matter more than any single number.

Hardware and software environments. Low-end hardware is approximated by the 360 px Galaxy S8 profile and Lighthouse's 4× CPU throttling. Three engines (Chromium, Gecko, WebKit) cover Android, iOS, Windows and macOS browsers. Frame-rate metrics are not applicable to this document-style app: animations are CSS-only, layout shift was measured by Lighthouse, and reduced motion removes the confetti effect. One limitation remains: devices are emulated, not physical, and testing on peers' own phones is the planned complement.

4. Code Quality, OOP and Algorithms

Structure. A monorepo with *client/* (app routes, reusable components, typed API client) and *server/*. The server is split into *routes* (HTTP only), *services* (business logic and SQL), *domain* (pure classes), *middleware*, *utils*, *content* and *migrations*. No file mixes HTTP, SQL and rules. Each zod schema is the single source of the validation rules and the TypeScript types.

Principle	Where it is applied
Encapsulation	*QuestionSelector* hides its random source and the shuffle; *StreakCalculator* keeps its rest-day rule private; services expose functions, not SQL
Abstraction, inheritance, polymorphism	Abstract *PaceSignal<T>* defines *evaluate()* once (template method); *ReadingPaceSignal* and *QuizPaceSignal* only override *expectedSeconds()*. *HttpError* and *InvalidPathError* extend *Error* and are handled polymorphically by one error handler
SOLID	Single responsibility (route, service and domain layers); open/closed (a new pace signal or crisis pattern list needs no changes elsewhere); dependency injection (the random generator, rest days and patterns are passed into constructors, so tests are deterministic); the *createApp()* factory lets tests run without a port

Principle	Where it is applied
Reusable components	UI kit (*Card, Button, Pill, Alert*), *StreakChip*, the lesson-block renderers and the *useApi* hook are shared across 20+ screens

```
// server/src/domain/learning-integrity.ts (abridged)
export abstract class PaceSignal<T> {
  protected constructor(private readonly fraction: number, private readonly floorSeconds: number)
  {} protected abstract expectedSeconds(work: T): number; // each signal says only this
  evaluate(work: T, actualSeconds: number): PaceResult { // shared template method
    const threshold = Math.max(this.floorSeconds, this.expectedSeconds(work) *
    this.fraction); return { ..., rushed: actualSeconds < threshold };
  }
}
export class ReadingPaceSignal extends PaceSignal<{ body: string }> { // rushed below 25% of reading
  time constructor() { super(0.25, 8); }
  protected expectedSeconds({ body }) { return (countWords(body) / 200) * 60; }
}
```

*Figure 4. Inheritance, polymorphism and encapsulation: services call **evaluate()** without knowing which signal they hold; the same pattern keeps quizpacing in **QuizPaceSignal**.*

Algorithm / custom solution	**Problem solved**	**Efficiency**
Fisher–Yates shuffle (questions and options)	Unbiased order, so answer positions can't be memorised	O(n)
Least-seen-first draw: shuffle, then stable sort by times seen	A "new quiz" really asks different questions, with ties broken at random	O(n log n)
Streak with a sliding 7-day rest-day window	Motivating but forgiving streak; best streak kept	O(days)
Pace threshold max(floor, fraction × words ÷ 200 wpm)	Detect rushing without blocking anyone	O(words)
Normalised gain g = (post − pre) ÷ (100 − pre) (Hake)	A fair learning measure whatever the starting score	O(1) per person
Scenario path replay over the script graph and link validation	Unforgeable insight scores; broken scripts caught at deploy	O(path), O(nodes + edges)
Group suppression (k ≥ 3); timing-safe login; LIKE escaping	Privacy in small cohorts; no username enumeration; no wildcard injection	O(1)
Version-aware, idempotent content seeding	Upgrade live content while keeping section ids and admin edits	O(sections)

Readability and maintainability. Names are consistent and meaningful (*markSectionViewed*, *timesSeen*, *MIN_GROUP_SIZE*), TypeScript runs in strict mode on both sides, ESLint runs on the client, and comments explain *why* rather than *what*. Documentation covers architecture, API, schema (ERD), deployment, security and testing. GitHub Actions type-checks, tests, lints and builds every push, then runs all journeys on 7 profiles plus Lighthouse and the load test.

5. Limitations and Next Steps

Partner media for the modules, live deployment with uptime measurement, and the NCSA authorisation or Rwandan hosting are the remaining steps before real participants use Beacon. Testing on physical phones, a moderated forum, peer pairing, Kinyarwanda and an encryption key-rotation script are future work.